

Decisiones de la Dirección del Proyecto - Sistema cineTP

Decisiones de la Dirección del Proyecto

Sistema cineTP · Documentación Técnica y Defensa de Proyecto

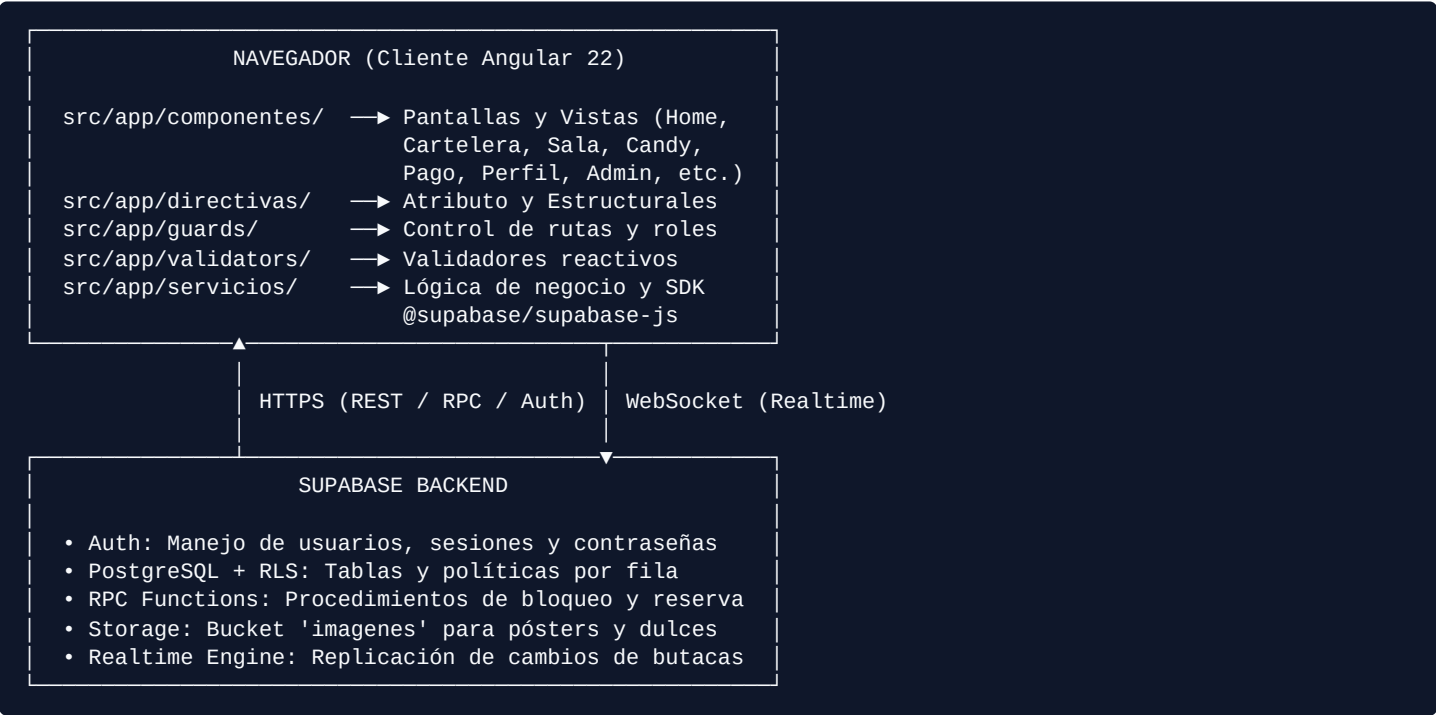
- **Proyecto:** cineTP
- **Framework:** Angular 22 (Standalone Components, Signals, Computed, Effects, SSR/CSR)
- **Backend:** Supabase (PostgreSQL, Auth, Row Level Security, Storage, RPC, Realtime WebSockets)
- **Despliegue:** Firebase Hosting · URL: <https://cinetp-ab899.web.app>

Índice

1. Justificación del enfoque elegido y decisiones de dirección
2. Diseño de entidades y modelado de datos (CRUD inicial)
3. El cuerpo del sistema: arquitectura CRUD y administración
4. Flujo inicial de navegación y rutas (app.routes.ts)
5. Reactividad Angular 22: Signals, Computed, Effects y Ciclo de Vida
6. Formularios reactivos y validaciones desacopladas (validators/)
7. Capa de servicios y persistencia con Supabase (servicios/)
8. Guards de navegación y directivas personalizadas
9. Pipes y formateo de datos en plantillas
0. Seguridad (RLS), Supabase Realtime y concurrencia

1. Justificación del enfoque elegido y decisiones de dirección

Elegimos esta arquitectura porque responde a los requerimientos reales del proyecto: una plataforma integral de cine comercial con flujo de compra interactivo, panel administrativo centralizado, validación de accesos con escaneo de códigos QR y control de roles. La prioridad de la dirección técnica fue lograr un balance óptimo entre **velocidad de desarrollo**, **claridad del modelo de dominio** y **alta mantenibilidad**, evitando la sobreingeniería y las abstracciones innecesarias.

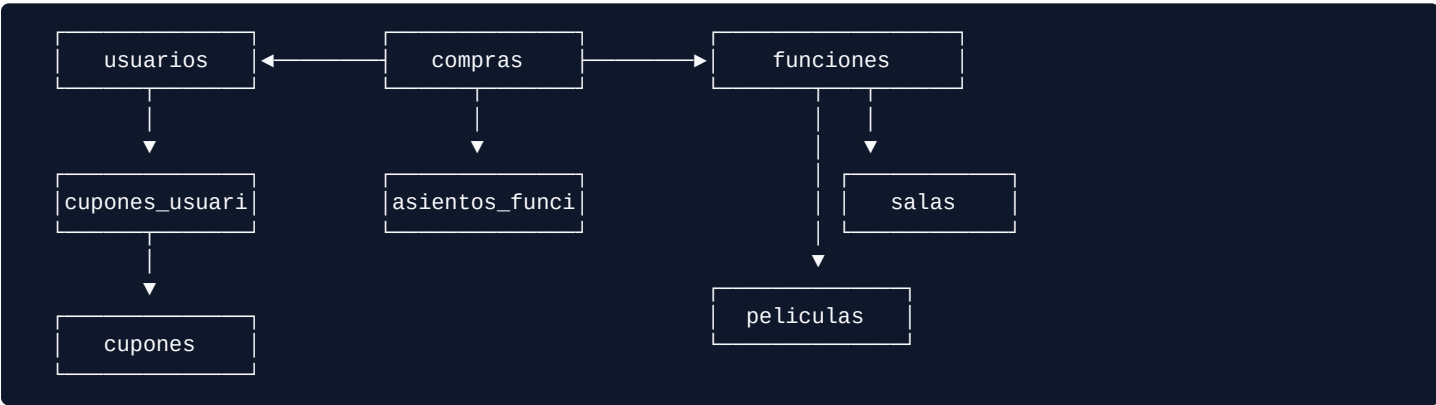


Principios rectores de la dirección técnica

- 1. **Modelado explícito de entidades de negocio:** Se rechazó la idea de un modelo genérico multipropósito. Cada concepto del negocio (`Película` , `Función` , `Sala` , `Producto Candy` , `Cupón` , `Usuario` , `Compra`) cuenta con su propia tabla relacional en PostgreSQL, su interfaz TypeScript tipada y su validador reactivo específico.
- 2. **Desacoplamiento entre Película y Función:** Una película representa la obra artística y sus metadatos (título, sinopsis, duración, géneros, restricción de edad, póster); una función representa la proyección concreta en un horario, sala, idioma y formato determinado. Separarlas elimina duplicación masiva de datos y permite modificar horarios, precios y salas sin alterar el catálogo general.
- 3. **Adopción de Angular 22 con Signals:** Se prescindió de los antiguos módulos (`NgModule`) en favor de componentes Standalone y Signals. Esto proporciona reactividad síncrona y de grano fino para filtros en vivo, carritos de confitería y selección de butacas sin la complejidad de suscripciones manuales de RxJS ni desuscripciones obligatorias.
- 4. **Supabase como Backend-as-a-Service (BaaS):** Delegar la infraestructura a PostgreSQL gestionado en Supabase permitió contar con autenticación robusta (JWT), almacenamiento multimedia (Storage para pósters), transacciones atómicas (RPC) y eventos en vivo (Realtime WebSockets) sin construir ni mantener una API REST intermedia desde cero.
- 5. **Validación en capas y seguridad centrada en la base:** Las validaciones en el frontend (`src/app/validators/`) brindan una excelente experiencia de usuario con mensajes inmediatos. Sin embargo, la seguridad definitiva descansa en la base de datos a través de **Row Level Security (RLS)** y restricciones relacionales.

2. Diseño de entidades y modelado de datos (CRUD inicial)

El diseño del modelo relacional en PostgreSQL se articuló en torno a las entidades que componen el flujo transaccional de compra y la gestión del cine.



2.1. Núcleo del flujo de compra y relaciones

La transacción principal del cliente consiste en seleccionar una **función específica** de una **película** proyectada en una **sala determinada**, pudiendo opcionalmente complementar su experiencia agregando productos del **Candy Bar** y aplicando **cupones promocionales**.

2.2. Estructura de tablas del backend (Supabase PostgreSQL)

Tabla	Clave Primaria / Tipo	Columnas Principales y Tipos	Relaciones y Propósito
<code>usuarios</code>	<code>id: uuid</code>	<code>email: text</code> , <code>nombre: text</code> , <code>apellido: text</code> , <code>fecha_nacimiento: date</code> , <code>rol: text</code> , <code>puntos: int4</code> , <code>credito: numeric</code> , <code>tipo_sangre: text</code> , <code>color_ojos: text</code> , <code>dias_vacaciones: int4</code>	Vinculada 1:1 con <code>auth.users.id</code> . Almacena datos del perfil y roles (<code>cliente</code> , <code>empleado</code> , <code>admin</code>).

Tabla	Clave Primaria / Tipo	Columnas Principales y Tipos	Relaciones y Propósito
peliculas	id: uuid	titulo: text, sinopsis: text, duracion: int4, formato: formato_pelicula, idioma: idioma_pelicula, generos: _genero_pelicula, restriccion_edad: int4, puntaje_compra: int4, poster: text, fecha_estreno: date, activa: bool, boletos_vendidos: int4, en_cartelera: bool	Catálogo de films. Control de visibilidad pública, ranking de ventas y restricción por edad.
salas	id: text	nombre: text, formato: text, capacidad_total: int4, activa: bool, creada_en: timestamptz	Salas de proyección. Identifica su geometría, formato (2D/3D) y aforo.
funciones	id: uuid	pelicula_id: uuid, sala_id: text, inicio: timestamptz, fin: timestamptz, precio: numeric, estado: estado_funcion, formato: varchar, idioma: varchar, es_preventa: bool	Proyecciones programadas. Claves foráneas a peliculas y salas. fin añade +30 min de limpieza.
asientos_funcion	(funcion_id, asiento_id)	usuario_id: uuid, estado: text, expira_en: timestamptz, sesion_id: uuid, sesion_hash: text	Matriz de butacas por función. Gestiona reservas temporales y ventas en tiempo real vía WebSockets.
productos_candy	id: uuid	nombre: varchar, descripcion: text, categoria: text, precio: numeric, imagen: text, puntaje_compra: int4, activo: bool, cantidad_vendida: int4	Artículos de confitería (combo, pochoclos, bebida, golosina).
cupones	id: uuid	codigo: varchar, nombre: text, descripcion: text, porcentaje_descuento: numeric, edad_minima_requerida: int4, es_primera_compra: bool, activo: bool	Promociones globales (ej. 20% primera compra, cupón mayores de 50 años).
cupones_usuario	id: uuid	usuario_id: text, cupon_id: uuid, usado: bool, asignado_en: timestamptz, usado_en: timestamptz	Control de unicidad de canjes de cupones por usuario registrado.

Tabla	Clave Primaria / Tipo	Columnas Principales y Tipos	Relaciones y Propósito
compras	id: uuid	codigo: text, usuario_id: text, funcion_id: text, pelicula_id: text, asientos: jsonb, total_entradas: numeric, items_candy: jsonb, total_candy: numeric, descuento: numeric, total_final: numeric, puntos_ganados: int4, qr_data: text, estado: text	Registro definitivo de venta. Guarda el payload codificado del QR y detalle desglosado.
auditoria	id: uuid	accion: text, detalle: text, usuario_id: text, usuario_email: text, creado_en: timestampz	Bitácora inmutable de acciones realizadas por administradores y empleados.
configuracion_cine	id: text	multiplicador_comun: numeric, multiplicador_vip: numeric, multiplicador_discapacitado: numeric, precio_base_referencia: numeric	Parámetros de tarificación para butacas comunes, discapacitados (filas J-K) y VIP (filas R-S-T).

2.3. Modelos de datos TypeScript (src/app/models/)

Cada tabla del backend cuenta con su interfaz tipada en Angular: - **Usuario (usuario.ts)**: id, email, nombre, apellido, fechaNacimiento, rol: RolUsuario (cliente | empleado | admin), puntos, credito. - **Pelicula (pelicula.ts)**: id, titulo, sinopsis, duracion, generos: GenerosPelicula[], restriccionEdad (0, 13, 16, 18), puntajeCompra, poster, fechaEstreno, enCartelera, boletosVendidos. - **Funcion (funcion.ts)**: id, peliculaId, salaId, inicio, fin, precio, estado: EstadoFuncion, formato: Formato (2D/3D), idioma: Idioma, esPreventa. - **Sala y Asiento (sala.ts)**: Matriz de 20 filas (A a T) con 3 bloques (4-20-4), tipos de fila (comun, discapacitado, vip) y capacidad total calculada. - **ProductoCandy (candy.ts)**: id, nombre, descripcion, precio, categoria: CategoriaCandy, stock, puntajeCompra, imagen. - **Cupon (cupon.ts)**: id, codigo, nombre, descripcion, porcentajeDescuento, edadMinimaRequerida, esPrimeraCompra, activo. - **Compra (compra.ts)**: id, codigo, usuarioId, funcionId, asientos, itemsCandy, totalEntradas, totalCandy, descuento, totalFinal, puntosGanados, qrData, qrImage, estado.

3. El cuerpo del sistema: arquitectura CRUD y administración

Para evitar duplicación de código en los múltiples formularios de gestión administrativa, se implementó un patrón de diseño basado en una **clase base abstracta genérica**: AdministrarBase<T> (src/app/componentes/admin/administrar-base.ts).

```

export abstract class AdministrarBase<T extends { id: string }> implements OnInit {
  protected abstract servicio: BaseSupabaseService<T>;

  // Estado reactivo unificado para todos los CRUD administrativos
  items = signal<T[]>([]);
  itemSeleccionado = signal<T | null>(null);
  itemEditando = signal<T | null>(null);
  modoModal = signal<'crear' | 'editar' | null>(null);
  cargando = signal<boolean>(false);
  guardando = signal<boolean>(false);
  subiendoImagen = signal<boolean>(false);
  mensajeError = signal<string | null>(null);
  mensajeExito = signal<string | null>(null);

  async ngOnInit(): Promise<void> {
    await this.cargarData();
  }

  async cargarData(): Promise<void> {
    this.cargando.set(true);
    try {
      this.items.set(await this.servicio.obtenerTodos());
    } catch (err) {
      this.mensajeError.set(err instanceof Error ? err.message : 'Error al cargar datos.');
```

```

    } finally {
      this.cargando.set(false);
    }
  }

  abrirCreacion(): void {
    this.itemEditando.set(null);
    this.modoModal.set('crear');
```

```

  }

  abrirEdicion(item: T): void {
    this.itemEditando.set(item);
    this.modoModal.set('editar');
```

```

  }

  cerrarEdicion(): void {
    this.modoModal.set(null);
    this.itemEditando.set(null);
  }
}

```

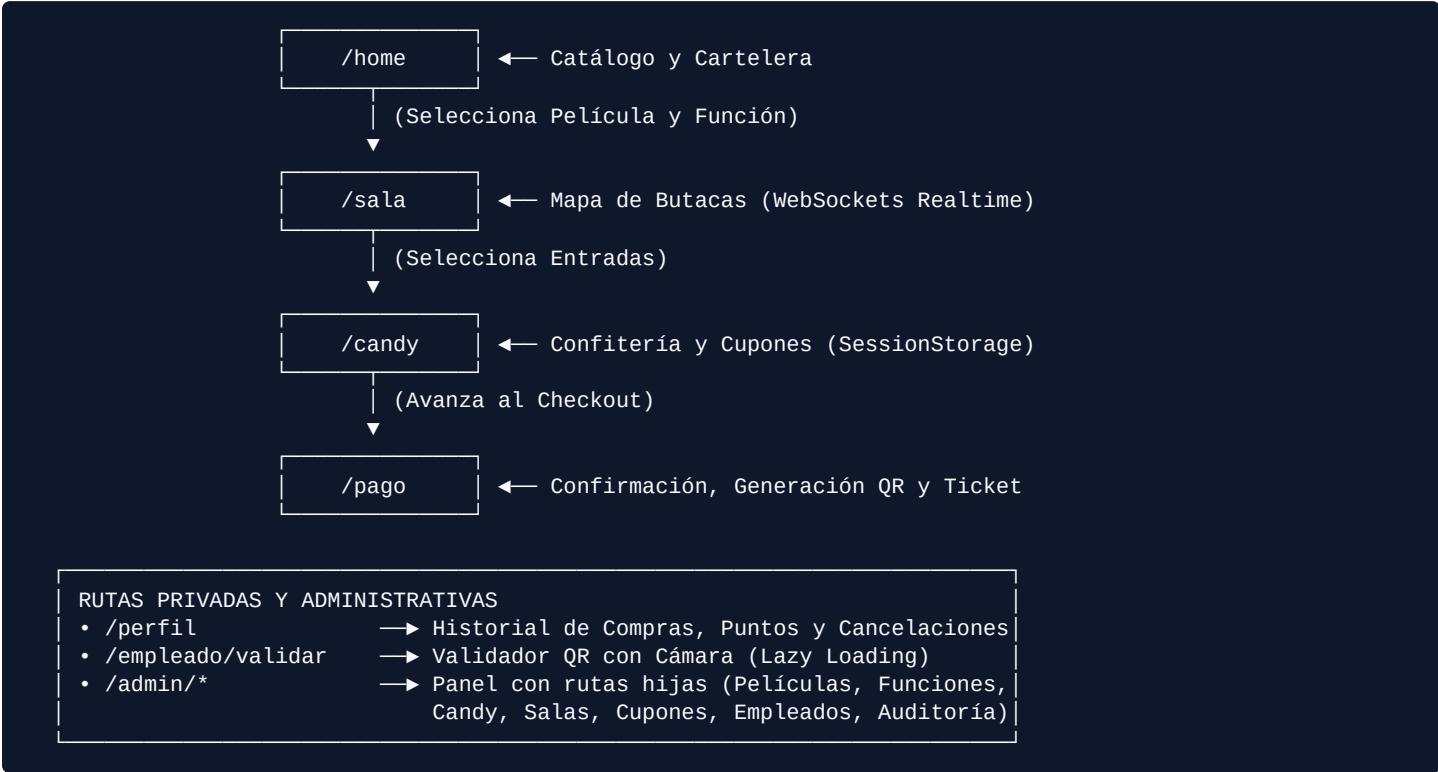
Implementaciones concretas del CRUD

- CRUD de Películas (AdministrarPelículas):** Hereda de `AdministrarBase<Película>`. Incorpora vista previa reactiva en tiempo real (`películaEditandoPreview`) mientras el administrador redacta la sinopsis o sube el póster vía `SupabaseStorageService`. Incluye búsqueda reactiva y alternancia de géneros.
- CRUD de Funciones (AdministrarFunciones):** Hereda de `AdministrarBase<Función>`. Carga listas auxiliares de películas y salas. Valida superposiciones horarias garantizando los 30 minutos obligatorios de limpieza entre proyecciones mediante `FuncionesService.validarSuperposicionHoraria()`.
- CRUD de Salas (AdministrarSalas):** Hereda de `AdministrarBase<Sala>`. Integra el componente `app-mapa-sala` pasando `[salaInput]="sala"` para previsualizar la disposición geométrica de las 20 filas (comunes, discapacitados en filas J-K y VIP en filas R-S-T).
- CRUD de Candy Bar (AdministrarCandy):** Hereda de `AdministrarBase<ProductoCandy>`. Gestiona categorías, precios, control de stock, puntos otorgados y subida de imágenes de productos.
- CRUD de Cupones (AdministrarCuponesComponent):** Hereda de `AdministrarBase<Cupon>`. Permite definir porcentajes de descuento, restricciones de edad mínima y validez de primera compra con vista previa reactiva.
- Alta de Empleados (RegistrarEmpleado):** Componente administrativo para el registro controlado de usuarios con rol `'empleado'`.

7. **Bitácora de Auditoría (Auditoría)**: Visualiza los registros inmutables de operaciones administrativas y de empleados consultando AuditoriaService .

4. Flujo inicial de navegación y rutas (app.routes.ts)

El enrutador de Angular (src/app/app.routes.ts) estructura la experiencia del usuario separando la navegación pública, el flujo de compra asistido, el perfil del cliente, las herramientas de empleado y el panel administrativo.



4.1. El paso a paso del flujo de compra

- Paso 1 (/home)**: El cliente ingresa al catálogo general (CarteleraComponent). Puede filtrar por título, género cinematográfico o formato (2D/3D), y consultar el Top 3 de más vistas. Al seleccionar una película, se despliega el listado de funciones disponibles. Al elegir un horario, el router navega a /sala?funcionId=... .
- Paso 2 (/sala)**: Se activa MapaSalaComponent . El componente lee el parámetro funcionId , carga las butacas ocupadas desde Supabase y abre un canal WebSocket de Realtime. El cliente selecciona sus asientos; si otro usuario reserva una butaca en ese instante, el mapa se actualiza en vivo. Al presionar “Continuar al Candy”, guarda las entradas en sessionStorage (cinetp_compra_actual) y navega a /candy .
- Paso 3 (/candy)**: Se activa SeleccionCandyComponent . El cliente agrega combos o golosinas y puede aplicar cupones promocionales (ej. 20% primera compra). Las propiedades computadas subtotalBruto , descuentoMonto y totalFinal se recalculan reactivamente. Al presionar “Continuar al Pago”, actualiza el contexto y navega a /pago .
- Paso 4 (/pago)**: PagoComponent toma el contexto, llama a ComprasService.crearCompra() para persistir la compra en Supabase, genera el código alfanumérico y la imagen QR interactiva con la librería qrcode , confirma los asientos y acredita puntos. El ticket queda renderizado en pantalla.
- Paso 5 (/perfil)**: Desde su perfil, el cliente puede consultar todas sus compras pasadas con su código QR para el ingreso, su saldo acumulado de puntos y créditos, y solicitar cancelaciones hasta 2 horas antes de la función con reintegro automático en crédito.

4.2. Definición formal de rutas (`src/app/app.routes.ts`)

```
export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'login', component: Login, canActivate: [authGuard] },
  { path: 'registro', component: Registro, canActivate: [authGuard] },
  { path: 'register', redirectTo: 'registro', pathMatch: 'full' },
  { path: 'home', component: Home },
  { path: 'perfil', component: PerfilComponent, canActivate: [authGuard] },
  { path: 'sala', component: MapaSalaComponent },
  { path: 'candy', component: SeleccionCandyComponent },
  { path: 'pago', component: PagoComponent },
  {
    path: 'empleado/validar',
    // Carga diferida (Lazy Loading): el lector de QR solo se descarga para el personal
    loadComponent: () =>
      import('./componentes/empleado/validar-entrada/validar-entrada').then((m) =>
m.ValidarEntrada),
    canActivate: [authGuard, roleGuard],
    data: { roles: ['empleado', 'admin'] },
  },
  {
    path: 'admin',
    component: Admin,
    canActivate: [authGuard, roleGuard],
    data: { roles: ['admin'] },
    children: [
      { path: '', redirectTo: 'peliculas', pathMatch: 'full' },
      { path: 'peliculas', component: AdministrarPeliculas },
      { path: 'funciones', component: AdministrarFunciones },
      { path: 'candy', component: AdministrarCandy },
      { path: 'salas', component: AdministrarSalas },
      { path: 'cupones', component: AdministrarCuponesComponent },
      { path: 'empleados/nuevo', component: RegistrarEmpleado },
      { path: 'auditoria', component: Auditoria },
    ],
  },
  { path: '**', redirectTo: 'login' },
];
```

5. Reactividad Angular 22: Signals, Computed, Effects y Ciclo de Vida

El proyecto aprovecha plenamente las capacidades reactivas modernas de Angular:

- `signal<T>()` / `WritableSignal<T>`: Estado mutable síncrono. Se utiliza para listas de datos, banderas de carga (`cargando = signal(true)`), textos de búsqueda y errores.
- `computed()`: Valores derivados reactivos de solo lectura. Se recalculan automáticamente únicamente cuando cambian sus dependencias (ej. `topTresVendidas`, `peliculasFiltradas`, `totalCandy`, `totalFinal`).
- `effect()`: Reacciones secundarias que se ejecutan automáticamente cuando cambian los signals observados (ej. las directivas `*appAdmin` y `*appEmpleado` que agregan o eliminan nodos del DOM al cambiar `usuarioActual()`).
- **Ciclo de Vida (`ngOnInit` y `ngOnDestroy`):**
 - `ngOnInit`: Utilizado en todas las pantallas principales para la hidratación asíncrona de datos desde Supabase mediante `async/await`.
 - `ngOnDestroy`: Fase crítica de limpieza para liberar hardware de cámara (`stream.getTracks().forEach(t => t.stop())` en `validar-entrada.ts`), detener timers (`clearInterval` en `mapa-sala.ts`) y desconectar canales WebSockets de Supabase Realtime, previniendo fugas de memoria.
 - `ngOnChanges`: Prescindible en este enfoque porque los **Signal inputs** (`input()`) combinados con `computed()` se recalculan solos.

6. Formularios reactivos y validaciones desacopladas (`validators/`)

Las validaciones del sistema residen en funciones puras y desacopladas bajo `src/app/validators/`:

- 1. **Autenticación** (login.ts , registro.ts): Validación estricta de formato email mediante Regex, longitud mínima de contraseñas (6 caracteres), coincidencia obligatoria de contraseña (passwordsMatchValidator) y validación de fecha de nacimiento (fechaNacimientoValidator) impidiendo fechas futuras o menores de edad no permitidos.
- 2. **Películas** (pelicula.ts): Título (mín. 2 letras), sinopsis (mín. 10 caracteres), duración (1 a 600 min), restricción de edad (0 a 18) y función constructora con vista previa en tiempo real (generarPreviewPelicula).
- 3. **Funciones** (funcion.ts): Selección obligatoria de película, sala, fecha, hora y precio base mayor a \$0.
- 4. **Candy Bar** (candy.ts): Nombre, categoría obligatoria, precio no negativo, stock y puntos de fidelidad.
- 5. **Salas** (sala.ts): Nombre y formato; genera la vista previa calculando la capacidad total de asientos.
- 6. **Cupones** (cupon.ts): Código en mayúsculas, descripción y porcentaje de descuento válido (1% a 100%).

7. Capa de servicios y persistencia con Supabase (servicios/)

Los servicios encapsulan la lógica de negocio y las llamadas a la base de datos, declarándose como singletons con @Injectable({ providedIn: 'root' }):

Servicio	Archivo	Responsabilidad Principal
Auth	servicios/auth.ts	Sesión con Supabase Auth (signUp , signInWithPassword , signOut), Signals globales usuarioActual y cargando , sincronización de puntos y crédito.
SupabaseClientService	servicios/supabase-client.ts	Inicializa la instancia única del cliente @supabase/supabase-js con la URL y la clave pública anónima.
BaseSupabaseService<T>	servicios/base-supabase.ts	Clase abstracta con métodos CRUD reutilizables (obtenerTodos , obtenerPorId , insertar , actualizar , eliminar) y mapeo a snake_case.
PeliculasService	servicios/peliculas.ts	Catálogo cinematográfico, filtro de cartelera, cálculo de Top 3 e incremento atómico de boletos vendidos.
FuncionesService	servicios/funciones.ts	Programación de funciones, cálculo de horario fin (+30 min de limpieza) y detección de choques de sala.
SalaService	servicios/sala.ts	Geometría de salas: genera 20 filas (A-T) con 3 bloques (4-20-4), discapacitados (filas J-K) y VIP (filas R-S-T).
CandyService	servicios/candy.ts	Artículos de confitería, filtrado por categorías, stock y puntos asociados.
CuponesService	servicios/cupones.ts	Promociones, validación de cupón de bienvenida (20%), cupón +50 años y control de edad mínima.
ComprasService	servicios/compras.ts	Orquestación de compra, generación de QR con qrcode , confirmación de asientos, crédito por cancelación a 2 hs.

Servicio	Archivo	Responsabilidad Principal
AsientosRealtimeService	servicios/asientos-realtime.ts	Conexión WebSocket a <code>asientos_funcion</code> y llamadas a RPCs de reserva y confirmación.
AuditoriaService	servicios/auditoria.ts	Registro inmutable de eventos administrativos y de empleados en la tabla <code>auditoria</code> .
SupabaseStorageService	servicios/supabase-storage.ts	Carga de imágenes en el bucket <code>imagenes</code> , previsualización con <code>FileReader</code> y obtención de URL pública.

8. Guards de navegación y directivas personalizadas

Guards de Navegación Funcionales (`CanActivateFn`)

- `authGuard` (`src/app/guards/auth-guard.ts`): Ejecuta `await authService.obtenerUsuario()` para esperar la resolución asíncrona de la sesión en el navegador (evita rebotes en falso al recargar F5). Si no hay sesión, redirige a `/login?returnUrl=...`. Si ya está logueado e intenta ir a login/registro, lo redirige a la pantalla de su rol (`rutaInicioPorRol`).
- `roleGuard` (`src/app/guards/role-guard.ts`): Lee `route.data['roles']`. Si un cliente intenta ingresar a `/admin` o a `/empleado/validar`, lo redirige a la ruta autorizada para su rol.

Directivas Personalizadas

- **Directiva de Atributo:** `HoverZoomDirective` (`[appHoverZoom]`): Inyecta `ElementRef` y `Renderer2`. Escucha eventos con `@HostListener('mouseenter')` y `('mouseleave')`, aplicando una animación suave de escalado CSS (`transform: scale(1.05)`) en tarjetas de películas y dulces.
- **Directivas Estructurales:** `AdminDirective` (`*appAdmin`) y `EmpleadoDirective` (`*appEmpleado`): Inyectan `TemplateRef` y `ViewContainerRef`. Contienen un `effect()` reactivo que observa `auth.usuarioActual()`. Si el rol coincide, insertan la vista con `createEmbeddedView()`; en caso contrario, vacían el contenedor con `clear()`, retirando los elementos del DOM de raíz.

9. Pipes y formateo de datos en plantillas

Los **Pipes** transforman datos directamente en las plantillas sin mutar los modelos subyacentes:

- **Pipes Puros vs Impuros:** Los pipes puros (`pure: true`) solo se recalculan cuando cambia el valor primitivo o la referencia en memoria del argumento, aprovechando la **memoización** para no sobrecargar el ciclo de renderizado. Los impuros se ejecutan en cada ciclo de detección de cambios.
- **Pipes nativos en `cineTP`:** Se utiliza `uppercase` para estandarizar categorías de confitería y códigos de cupón, y `titlecase` para estados de función (`Programada`, `Cancelada`) e idiomas. Para fechas, se emplean utilitarios TypeScript basados en `Intl.DateTimeFormat` (`es-AR`) para garantizar el estándar argentino.

10. Seguridad (RLS), Supabase Realtime y concurrencia

Seguridad por Filas (Row Level Security - RLS)

La presencia de la clave `publishableKey` en `environments.ts` es pública por diseño en cualquier SPA. La seguridad real no depende de esconder la clave, sino de las políticas de RLS en PostgreSQL: cada consulta viaja con el token JWT del usuario, y la base de datos evalúa `auth.uid()` para restringir accesos. Los clientes solo pueden leer y gestionar sus propias compras y cupones, mientras que la modificación de películas, funciones y salas está bloqueada para cualquiera que no sea administrador.

Supabase Realtime y WebSockets

En lugar de saturar el servidor con peticiones repetitivas de sondeo (*polling* HTTP), `AsientosRealtimeService` abre un canal WebSocket bidireccional sobre la tabla `asientos_funcion`. Cuando cualquier cliente reserva o compra una butaca, Supabase replica el evento instantáneamente y las pantallas de todos los usuarios conectados actualizan el estado visual del mapa en milisegundos.

Resolución de Concurrencia (Race Conditions)

Realtime es una capa de notificación visual optimista; la consistencia transaccional la asegura PostgreSQL. Cuando dos usuarios intentan confirmar la misma butaca exactamente en el mismo milisegundo, la base de datos resuelve el conflicto atómicamente mediante procedimientos RPC (`confirmar_asientos`) o la restricción `UNIQUE (funcion_id, asiento_id)`. La primera transacción adquiere el bloqueo y tiene éxito; la segunda es rechazada por colisión en la base de datos, notificando al segundo usuario en pantalla que la butaca ya fue vendida.